WEEK 4 · DAY 3 · RULE BOOK

Give the Model Eyes: Feature Engineering

Follow it step by step. Every common error and its fix is here.

Your goal today: build the columns that let a model see what an engineer sees: **history** (lags, rolling windows), **relationships** (`temp_per_drive`), and then cut 32 columns down to 5. **15 green ✅ ticks** to collect. Time: about 55 minutes, then Checkpoint 3. Keep this page open next to Colab.

Before you start (2 minutes)

- ☐ Open `W4D3-exercise` in Colab → **File** → **Save a copy in Drive**.
- ☐ Run **STEP 0**. Wait for ✅ `Ready!`
- ☐ Run every ▶️ cell in order. Each ✍️ cell depends on the cells above it.
- ☐ The 🧰 **toolbox** cell at the start of Part B must run before anything after it.

The three kinds of eyes

Eye	The question it answers	Today's example
🕰️ **History**	What was it an hour ago? Over the last day?	`soc.shift(1)` , `soc.rolling(24).mean()`
🌘 **Context**	Which state is the satellite in?	`steady_sun(bus)` : no eclipse this hour or the 3 before
🔗 **Relationship**	Is it normal **for** the other sensor?	`temp_per_drive = twta_temp_c / rf_drive_pct`

The 14:00 test (ask it of every feature)

"Could I compute this at 14:00, using only what I knew at 14:00?" If not, delete it.

✅ Honest (past only)	❌ Cheats (peeks at the future)
`x.shift(1)` , `x.shift(24)`	`x.shift(-1)` (copies the NEXT hour back)
`x.diff(1)`	`x.rolling(24, center=True)` (mixes in 12 future hours)
`x.rolling(24).mean()` , `.std()`	anything fitted on the whole year, then judged on the same year

A 24-hour lag or window has nothing to look at during the first day, so `dropna()` removes those rows: 8,760 → 8,736.

The exercise, step by step

Task	What you do	The tool you need
A1	SoC one hour ago	`HOURS_BACK = 1`
A2	Average of the last day	`WINDOW = 24`
A3	Which candidate features cheat?	a set of letters, e.g. `{"a", "c"}`
A4	Rows lost by the 32-column builder	`len(bus) − len(F)`
B1	Build the relationship	`bus["twta_temp_c"] / bus["rf_drive_pct"]`
B2	3σ chart on `temp_per_drive`	`K = 3`
B3	EV04 hours lit by that chart	read `hours_flagged` in the event table
B4	Oct-Dec hours flagged by the 29σ feature	`resid_flags.loc["2025-10-01":].sum()`
B5	Does a bigger sigma always win?	`True` or `False` (no quotes)
C1	Count the twin columns	`CUTOFF = 0.98`
C2	Forest on all 32 columns	`CONTAMINATION = 0.005`
C3	Forest on the lean 5	`FIT_ROWS = train & steady`
C4	Why was EV04 flagged?	the winning column name, in quotes
C5	Is 5 of 5 on the whole year an honest exam?	`True` or `False`
🌶️ **Bonus**	Race the Week 4 chart with the same eyes	`CHART_FIT_ROWS = train & steady`

Today's tools (from the 🧰 toolbox cell)

```
steady = steady_sun(bus)                          # True in steady sunlight
X = lean_features(bus)                            # 4 sensors + temp_per_drive
flags = forest_flags(X, train & steady, steady)   # isolation forest: fit on the
reference, judge steady hours
flags = chart_flags(X, train & steady, steady)    # Week 4 chart on every column: any |z|
> 3
score_events(flags, labels)                       # caught, false alarms, recall,
precision
event_table(flags, labels)                        # per event: caught? hours late? hours
lit?
```

`train` = every hour before 1 September (January–August), the **reference months**. `train & steady` = reference months **and** steady sun.

Explain every flag: the z-score

```
z = (X - X[train & steady].mean()) / X[train & steady].std()
```

z = how many σ a feature sits from its normal. The biggest |z| on a flagged hour is your reason. Say it to the duty engineer: *"temp_per_drive is 3.8σ high: the amplifier is hot for its drive level."*

When things go wrong: error → fix

You see	It means	Fix
`AssertionError: ✍️ Fill in … first`	A `___` is still there	Replace it, run the cell again
`AssertionError: ✍️ Finish B1 first`	An earlier task is still blank	Go back and finish that task
`NameError: name 'steady_sun' …` / `'score_events'`	You skipped the toolbox	Run the 🧰 toolbox cell at the start of Part B
`NameError: name 'F' is not defined`	You skipped the builder	Run the `make_features` cell in Part A
B1 values around **1.2**	You divided drive by temperature	Temperature on top: `twta_temp_c / rf_drive_pct`
`ValueError: Invalid frequency: H`	Old code with capital `"H"`	Use lowercase `"h"`
Your false-alarm count differs a little from your neighbour's (C2, C3)	Different pandas or scikit-learn versions	Normal for the forest; the check allows a range
C2 says contamination is wrong / thousands of flags	You wrote a percentage (`5` , `0.5`)	It's a fraction of hours: `0.005`
Forest cells feel slow	200 trees on thousands of rows	A few seconds is normal
Everything is weird	–	**Runtime → Restart session and run all**

Rules of thumb you'll keep forever

1. **The model cannot tell time.** If history matters, put it in a column.
2. **Past only.** Every feature passes the 14:00 test, or it goes.
3. **Model the relationship, not the raw value**, when the raw value swings with the load or the season.
4. **A big sigma on the fault is not the test.** The test: does the feature stay quiet on normal months it never saw?

5. **Fewer, sharper features.** Prune twins above 0.98. Every column should answer an engineer's question.
6. **Explain every flag.** Right answer, right reason.

🏁 **Finished?** Run `scoreboard()`. Try the 🌶️ race, then get your Checkpoint 3 list ready for the trainer.